

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и автоматизированных
систем

Лабораторная работа №16

по дисциплине: Основы программирования

тема: **«Работа с многомерными массивами»**

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич

Проверили:
ст. преп. Притчин Иван Сергеевич
асс. Черников Сергей Викторович

Белгород 2023 г.

Цель работы: получение навыков работы с многомерными массивами.

Содержание отчета:

Тема лабораторной работы.

Цель лабораторной работы.

Решения задач:

– Текст задания.

– Исходный код (в том числе и тестов).

Вариант 6

Задание 5 Дана квадратная матрица. Если среди сумм элементов строк матрицы нет равных, то транспонировать матрицу

```
#include <stdio.h>
#include <assert.h>
#include "directoryOfMatrix/matrix.h"
#include <malloc.h>
#include <math.h>

bool isUnique(Matrix m, int n) {
    int *criteria = (int *) malloc(sizeof(int) * n);
    for (int i = 0; i < n; i++) {
        criteria[i] = getSum(m.values[i], m.nCols);
    }

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (criteria[i] == criteria[j]) {
                return false;
            }
        }
    }

    free(criteria);
    return true;
}

void transposeIfMatrixHasNotEqualSumOfRows(Matrix m) {
    assert(isSquareMatrix(m));

    if (isUnique(m, m.nRows)) {
        transposeSquareMatrix(m);
    }
}

int main() {
    Matrix m1 = createMatrixFromArray((int[]) {
        1, 2, 3,
        4, 5, 6,
        10, 11, 12,
    }, 3, 3);

    transposeIfMatrixHasNotEqualSumOfRows(m1);

    outputMatrix(m1);

    return 0;
}
```

Задание 9 Дано n точек в m -мерном пространстве. Упорядочить точки по неубыванию их расстояний до начала координат

```
#include <stdio.h>
#include <assert.h>
#include "directoryOfMatrix/matrix.h"
#include <malloc.h>
#include <math.h>

float getDistance(int *a, int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        count += a[i] * a[i];
    }

    return sqrt(count);
}

void swapF(float *a, float *b) {
    float t = *a;
    *a = *b;
    *b = t;
}

void insertionSortRowsMatrixByRowCriteriaF(Matrix m, float (*criteria)(int *,
int)) {
    float *criteriaArray = (float *) malloc(sizeof(float) * m.nRows);
    for (int i = 0; i < m.nRows; i++) {
        criteriaArray[i] = criteria(m.values[i], m.nCols);
    }

    for (int i = 0; i < m.nRows; i++) {
        int minIndex = i;
        for (int j = i + 1; j < m.nRows; j++) {
            if (criteriaArray[j] <= criteriaArray[minIndex]) {
                minIndex = j;
            }
        }

        if (minIndex != i) {
            swapF(&criteriaArray[i], &criteriaArray[minIndex]);
            swapRows(m, i, minIndex);
        }
    }

    free(criteriaArray);
}

int main() {
    Matrix m1 = createMatrixFromArray((int[]) {
        14, 5, 0,
        0, 0, 0,
        3, 3, 3,
    }, 3, 3);

    int b[3] = {1, 2, 3};

    insertionSortRowsMatrixByRowCriteriaF(m1, getDistance);

    outputMatrix(m1);

    return 0;
}
```

Задание 11 Дана матрица. Определить k – количество "особых" элементов матрицы, считая элемент "особым", если он больше суммы остальных элементов своего столбца

```
#include <stdio.h>
#include <assert.h>
#include "directoryOfMatrix/matrix.h"
#include <malloc.h>
#include <math.h>

int getNSpecialElement(Matrix m) {
    int *mas = (int *) malloc(sizeof(int) * m.nRows);
    for (int i = 0; i < m.nCols; i++) {
        mas[i] = getSum(m.values[i], m.nRows);
    }

    int count = 0;
    for (int i = 0; i < m.nCols; i++) {
        for (int j = 0; j < m.nRows; j++) {
            mas[i] -= m.values[i][j];
            if (mas[i] < m.values[i][j]) {
                count++;
                mas[i] += m.values[i][j];
            } else {
                mas[i] += m.values[i][j];
            }
        }
    }

    free(mas);
    return count;
}

int main() {
    Matrix m1 = createMatrixFromArray((int[]) {
        6, 1, 3,
        4, 1, 6,
        10, 11, 12,
    }, 3, 3);

    printf("%d", getNSpecialElement(m1));

    return 0;
}
```

Задание 12 Дана квадратная матрица. Заменить предпоследнюю строку матрицы первым из столбцов, в котором находится минимальный элемент матрицы.

```
#include <stdio.h>
#include <assert.h>
#include "directoryOfMatrix/matrix.h"
#include <malloc.h>
#include <math.h>

Position getLeftMin(Matrix m) {
    int iMin = 0;
    int jMin = 0;

    for (int i = 0; i < m.nRows; i++) {
        for (int j = 0; j < m.nCols; j++) {
            if (m.values[i][j] < m.values[iMin][jMin]) {
                iMin = i;
                jMin = j;
            }
        }
    }

    return (Position) {iMin, jMin};
}

void swapPenultimateRow(Matrix m, int n) {
    int *swapCol = (int *) malloc(sizeof(int) * m.nRows);
    for (int i = 0; i < m.nRows; i++) {
        swapCol[i] = m.values[i][n];
    }

    swapWithPointer(&swapCol, &m.values[m.nRows - 2]);
    free(swapCol);
}

int main() {
    Matrix m1 = createMatrixFromArray((int[]) {
        6, 3, 3,
        4, 5, 6,
        10, 1, 0,
    }, 3, 3);

    Position minPos = getLeftMin(m1);

    swapPenultimateRow(m1, minPos.colIndex);

    outputMatrix(m1);

    return 0;
}
```

Задание 17 Каждая строка данной матрицы представляет собой координаты вектора в пространстве. Определить, какой из этих векторов образует максимальный угол с данным вектором v .

```
#include <stdio.h>
#include <assert.h>
#include "directoryOfMatrix/matrix.h"
#include <malloc.h>
#include <math.h>

double getVectorLength(int *a, int n) {
    double length = 0;
    for (int i = 0; i < n; i++) {
        length += a[i] * a[i];
    }

    return sqrt(length);
}

double getCosine(int *a, int *b, int n) {
    return (a[0] * b[0] + a[1] * b[1] + a[2] * b[2]) / (getVectorLength(a, n) +
getVectorLength(b, n));
}

int getVectorIndexWithMaxAngle(Matrix m, int *b) {
    double *arrayOfCosines = (double *) malloc(sizeof(double) * m.nRows);
    for (int i = 0; i < m.nCols; i++) {
        arrayOfCosines[i] = getCosine(m.values[i], b, m.nRows);
    }

    double maxCosines = 0;
    int indexArray = 0;
    for (int i = 0; i < m.nRows; i++) {
        if (maxCosines < arrayOfCosines[i]) {
            maxCosines = arrayOfCosines[i];
            indexArray = i;
        }
    }

    free(arrayOfCosines);
    return indexArray;
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

int main() {
    Matrix m1 = createMatrixFromArray((int[]) {
        4, 5, 6,
        6, 3, 3,
        10, 1, 0,
    }, 3, 3);

    int b[3] = {1, 2, 3};

    printf("vector %d", getVectorIndexWithMaxAngle(m1, b));
    printf(" = ");
    printf("[ ");

    outputArray(m1.values[getVectorIndexWithMaxAngle(m1, b)], m1.nRows);

    printf("]");

    return 0;
}
```

Тесты функций

Задание 5

```
void test1_transposeIfMatrixHasNotEqualSumOfRows () {
    Matrix m1 = createMatrixFromArray((int[]) {
        1, 5, 4,
        1, 1, 2,
        4, 3, 2,
    }, 3, 3);

    transposeIfMatrixHasNotEqualSumOfRows(m1);

    Matrix m2 = createMatrixFromArray((int[]) {
        1, 1, 4,
        5, 1, 3,
        4, 2, 2,
    }, 3, 3);

    assert(areTwoMatricesEqual(m1,m2));
}

void test2_transposeIfMatrixHasNotEqualSumOfRows () {
    Matrix m1 = createMatrixFromArray((int[]) {
        1, 5, 4,
        1, 1, 2,
        4, 3, 2,
        1, 2, 3
    }, 4, 3);

    transposeIfMatrixHasNotEqualSumOfRows(m1);

    Matrix m2 = createMatrixFromArray((int[]) {
        1, 1, 4, 1,
        5, 1, 3, 2,
        4, 2, 2, 3
    }, 3, 4);

    assert(areTwoMatricesEqual(m1, m2));
}
```


Задание 12

```
void test1_getLeftMin() {
    Matrix m1 = createMatrixFromArray((int[]) {
        1, 5, 4,
        1, 1, 0,
        4, 3, 2,
    }, 3, 3);

    Position getLeft = getLeftMin(m1);

    assert(getLeft.colIndex == 2 && getLeft.rowIndex == 1);
}

void test2_getLeftMin() {
    Matrix m1 = createMatrixFromArray((int[]) {
        1, 5, 4,
        1, 1, 15,
        4, 0, 2,
    }, 3, 3);

    Position getLeft = getLeftMin(m1);

    assert(getLeft.colIndex == 2 && getLeft.rowIndex == 1);
}
```